

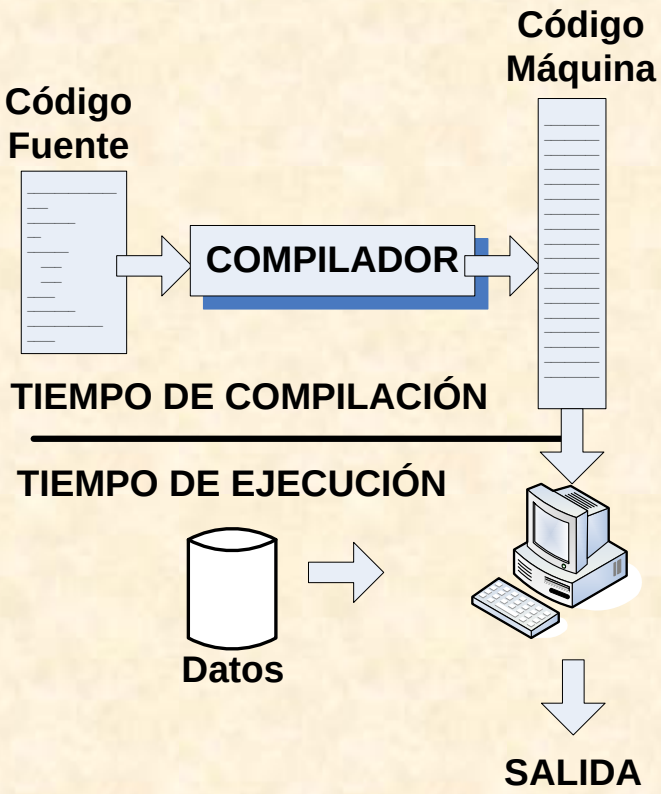
Compiladores



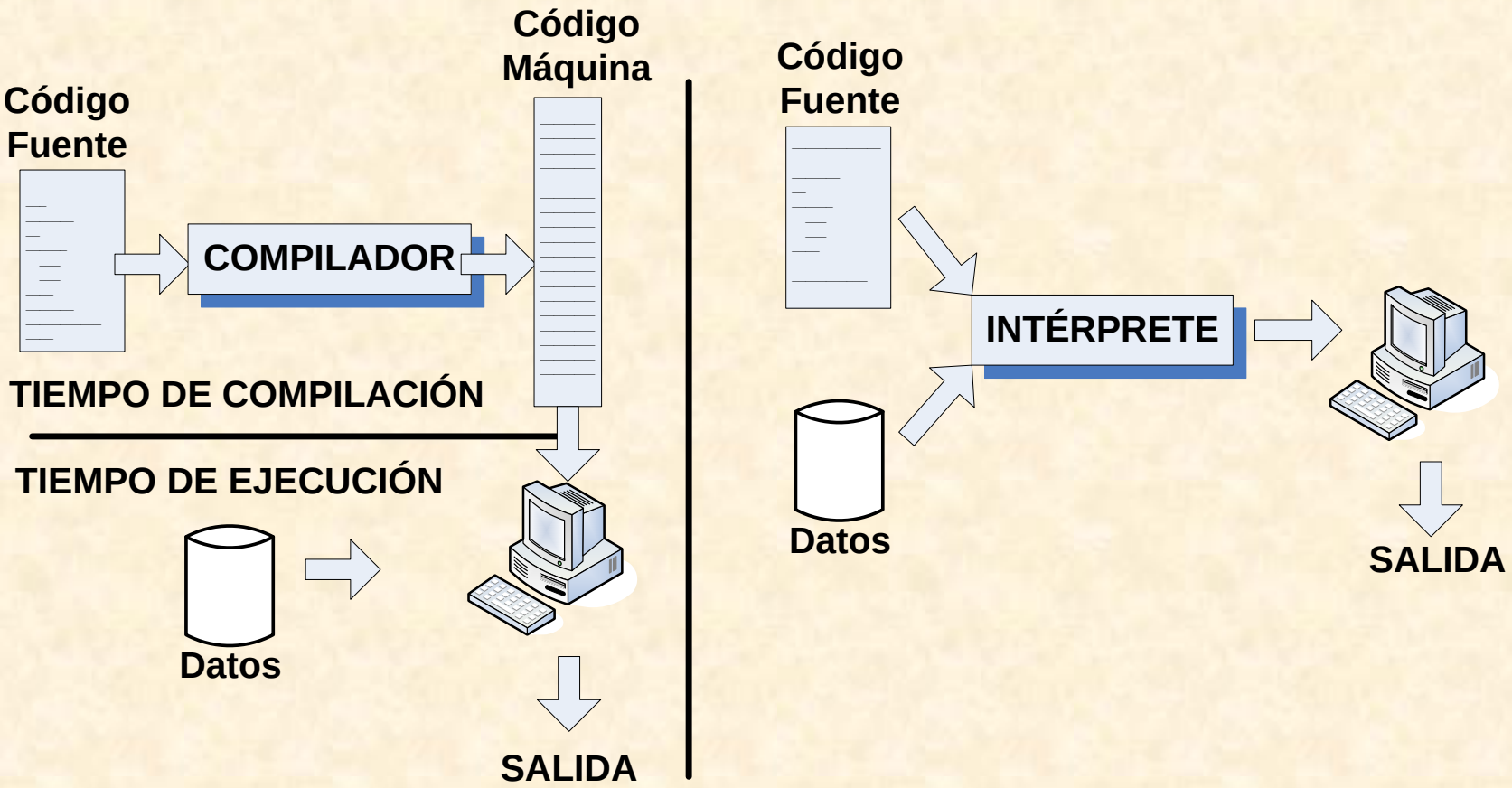
- Concepto: Programa que traduce o convierte un texto o programa escrito en un lenguaje fuente a un texto o programa equivalente escrito en un lenguaje destino, produciendo, si cabe, mensajes de error.

- Compiladores vs Intérpretes

Categorías de lenguajes
EJECUCIÓN



Categorías de lenguajes
EJECUCIÓN





Compilador vs Intérprete



Compilador

- Se obtiene un fichero ejecutable
- El proceso de compilación se realiza una vez
- Proceso más lento
- Mayor consumo memoria
- Mayor calidad en información sobre errores
- Mayor velocidad
- Menos errores en ejecución

Intérprete

- No se obtiene un fichero ejecutable
- El proceso de interpretación se realiza cada vez
- Proceso más rápido
- Menor consumo memoria
- Peor realimentación sobre errores
- Ejecución más lenta
- Errores en tiempo de ejecución



Conceptos



- Conceptos:
  - Lenguaje fuente
  - Lenguaje objeto
  - Lenguaje anfitrión



Especificación de sintaxis



- ¿Como podemos producir una especificación de la sintaxis de un lenguaje de programación?



Ejecicio en el aula



- Producir una especificación sintáctica para un lenguaje de programación que solo permite asignaciones con expresiones binarias y sentencia print



Ejecicio en el aula



- Producir una especificación sintáctica para la sección de declaración de variables. Solo se permiten los tipos int, float y string.



Ejecicio en el aula



- Extensión:
  - Permitir inicializaciones en línea. Por ejemplo:

int x=0,y=1,z;



Fases



- Análisis léxico
- Análisis sintáctico
- Análisis semántico
- Generación código intermedio
- Generación código objeto



Análisis léxico



- Análisis léxico
  - Descompone el código en tokens
  - Cada token contiene información sobre:
    - Tipo / Categoría
    - Posicionamiento en el código fuente original
    - Lexema



Ejemplos tokens



```
while (miVar<0) {
   miVar=2*miVar;
}
```



Ejemplos tokens



comision = fijo + precio * 8 ;



Ejemplos errores



```
X = Y * * 3
S = U V
T = 2 @ A
```



Especificación de tokens



- Mediante patrones léxicos a nivel de carácter
  - Diagramas de conway
  - Gramáticas lineales
  - Autómatas finitos
  - **Expresiones regulares**



Construcción de analizadores



- Herramientas de construcción de analizadores léxicos

$(\text{Expresión regular})_1$ $\{\text{acciones a ejecutar}\}_1$

$(\text{Expresión regular})_2$ $\{\text{acciones a ejecutar}\}_2$

...

$(\text{Expresión regular})_n$ $\{\text{acciones a ejecutar}\}_n$



Token/Patrón/Lexema



comision = fijo + precio * 8 ;

IDENTIFICADOR
1
1
comision

=
1
10

IDENTIFICADOR
1
12
fijo

○ ○ ○



Análisis Sintáctico



- Solicita tokens al analizador léxico y las organiza de acuerdo a las construcciones del lenguaje permitidas, formando el llamado árbol de análisis sintáctico
- Las construcciones permitidas vienen establecidas por la gramática del lenguaje, cuyos símbolos terminales son los que devuelve el analizador léxico



Análisis Semántico



- Cuando se detecta una construcción sintáctica, se realizan las comprobaciones semánticas pertinentes:
  - Comprobaciones de tipos
  - Declaraciones previas
  - …



Traducción



- En la traducción dirigida por la sintaxis, las herramientas permiten la introducción de acciones asociadas a las reglas de producción
- La implementación del generador de código intermedio se realiza insertando acciones en las partes derechas de las reglas de producción que acumulan la traducción parcial del subárbol en cada nodo.



Ejemplo



Lenguaje con:

- declaraciones (una por línea: int, float o string)
- sumas binarias de variables (en string concatena)
- lectura de variables con read
- mostrado de variables con print

Codigo de ejemplo:

```
int a;
int b;
read a;
read b;
a=a+b;
print a;
```



Ejemplo



- Lexemas:



Ejemplo



- Categorías Léxicas:
  - TIPO: float, int o string
  - PTOYCOMA
  - IDENTIFICADOR
  - IGUAL
  - NUMERO
  - CADENA
  - MAS
  - PRINT
  - READ



Ejemplo



(1) S -> lista_instrucciones

(2) lista_instrucciones -> instruccion PTOYCOMA lista_instrucciones
(3) -> instruccion PTOYCOMA

(4) instruccion -> declaracion
(5) -> asignacion
(6) -> lectura
(7) -> mostrado

(8) declaracion -> TIPO IDENTIFICADOR

(10) asignacion -> IDENTIFICADOR IGUAL IDENTIFICADOR MAS IDENTIFICADOR

(11) lectura -> READ IDENTIFICADOR

(12) mostrado -> PRINT IDENTIFICADOR

Codigo de ejemplo:

```
int a;
int b;
read a;
read b;
a=a+b;
print a;
```





- Derivar árbol sintáctico



Ejemplo tipos de errores



```
int x,y,$;
char t,2r;
while (x < 10){
    x++;
    t=2.0*t;
 }

whilr (y < 10){
   y++;
   t=2.0*t;
}
```



Ejemplo



```
int x,y,$;    // léxico
char t,2r;  // sintáctico
while (x < 10){
    x++;
    t=2.0*t;    // semántico
 }

whilr (y < 10){ // sintáctico
  y++;
  t=2.0*t; // semántico
}
```



Importancia de la tabla de símbolos



- Analizador semántico
  - Registra elementos declarados
  - Permite comprobación de tipos
- Generación de código intermedio
  - Permite saber la memoria necesaria
- Generación código final
  - Permite sustituir las referencias simbólicas a posiciones de memoria



Optimización de código



- Dependiente de la arquitectura
- El código intermedio permite producir código final para varias arquitecturas. La generación del código intermedio puede ser independiente de la arquitectura.
- Las optimizaciones y generación del código final pueden ser diferentes para cada arquitectura.
- El código final también puede ser optimizado (requiere conocimiento profundo del entorno de ejecución)



Algunas cuestiones



- Se trabaja con dos gramáticas
  - Análisis léxico
  - Análisis sintáctico

¿Hasta donde debe llevar cada una?

Maximizar simplicidad y naturalidad en las gramáticas